KP08 Speeding Up Backpropagation

> Level: undergraduate, year 3
>
> Pattern Recognition | Chapter 4, §4.7

Slides: read and download

Learning objectives

- Explain why a narrow valley makes gradient descent slow and swinging.
- Write the momentum update, work a few steps by hand, and explain why it speeds up the flat direction and cancels the steep one.
- State the rules of the adaptive learning rate and of quickprop and know the price and limits of each.

KP08 Speeding Up Backpropagation

One-sentence summary: Gradient descent is slow and swings in a narrow valley; momentum adds α times the previous step to this step, so the flat direction is enlarged and the steep one is cancelled; an adaptive learning rate lets μ adjust itself and quickprop estimates the valley floor from two slopes. Every speed-up costs extra complexity.

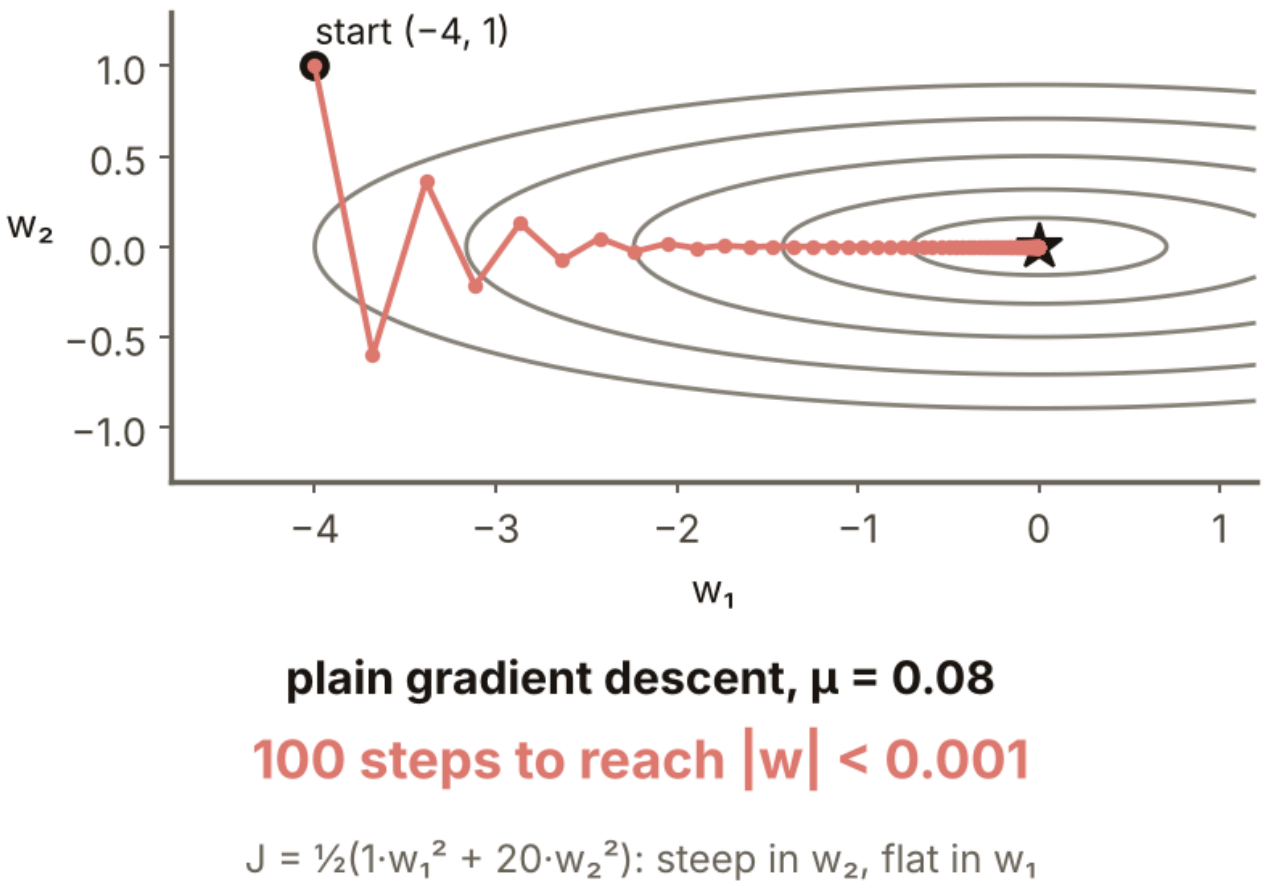
1. The narrow valley

Plain explanation: KP07 training took thousands of epochs, partly because gradient descent itself is slow.

- **Curvature**: how sharply the cost surface bends. Where the curvature is large, a small step changes the cost a lot.
- **Narrow valley (ill-conditioned surface)**: very large curvature in one direction and very small in another, so the contours are flat ellipses; the source calls it ill-conditioned, meaning the eigenvalues of the Hessian differ a lot.

Example: for this lecture we designed $J = \frac{1}{2}(w_1^2 + 20w_2^2)$ to show the mechanism: curvature 20 along w_2 and 1 along w_1, start $(-4, 1)$, $\mu = 0.08$. Plain gradient descent needs 100 steps to reach $|w| < 0.001$ and swings along w_2 (program).

Figure `lec05-kp08-valley`: contours and the path of plain gradient descent.

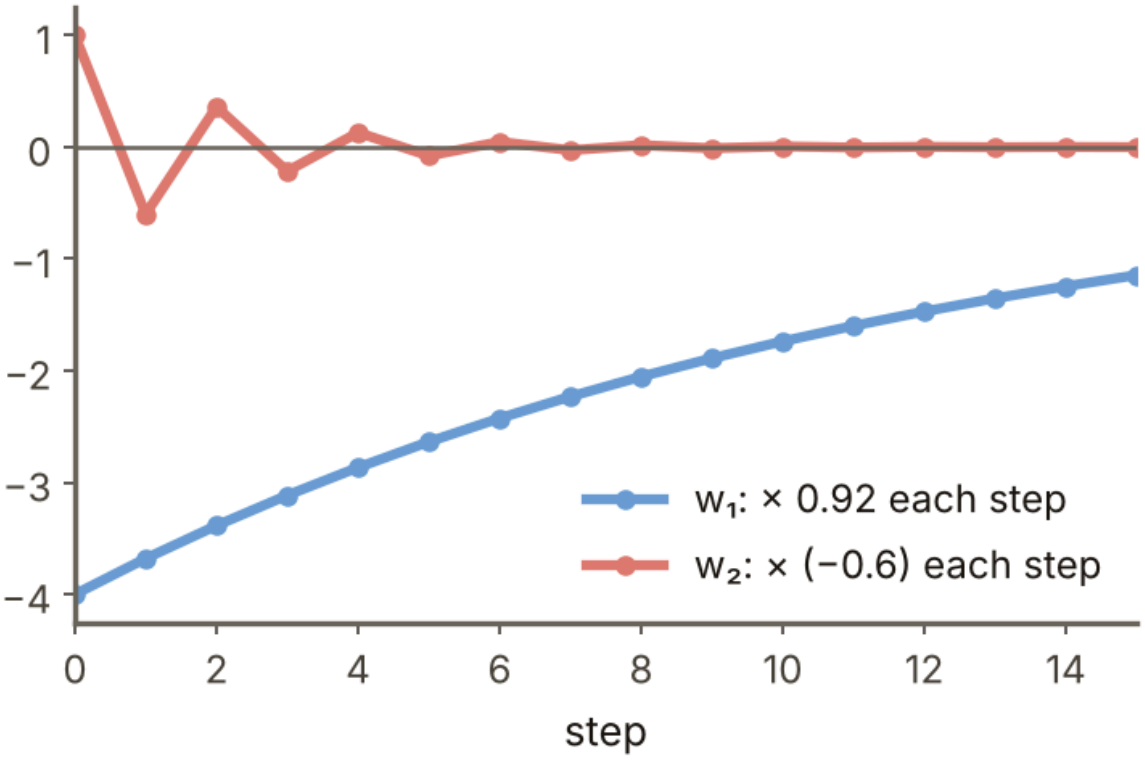


2. Why it swings

Steep direction: the gradient of w_2 is $20w_2$, so $w_2 \leftarrow w_2 - 0.08 \cdot 20\, w_2 = -0.6\, w_2$: it flips sign every step and shrinks only to 0.6 times. Flat direction: $w_1 \leftarrow w_1 - 0.08 \cdot 1 \cdot w_1 = 0.92\, w_1$, the same direction and only 8% smaller per step.

The dilemma: μ must stay below $2/20 = 0.1$ or w_2 swings ever wider; with $\mu = 0.12$ the norm $|w|$ exceeds 10^8 after 60 steps. But such a small step makes w_1 slow.

Figure `lec05-kp08-zigzag`: w_1 and w_2 over the first 15 steps.



1 − μ·20 = 1 − 0.08·20 = −0.6 1 − μ·1 = 0.92

μ must stay below 2/20 = 0.1 or w_2 blows up

3. The momentum term

Momentum: gives the update inertia: besides the new gradient step, this step adds α times the previous step.

Formal definition:

$$\Delta w(t) = \alpha\, \Delta w(t-1) - \mu\, g(t),$$

g is the gradient and α the momentum factor, 0.1 to 0.8 in practice; $\alpha = 0$ is plain gradient descent.

By hand ($\mu = 0.08$, $\alpha = 0.6$): step 1 is $(0.32, -1.6)$ and $w = (-3.68, -0.6)$; step 2 is $0.6 \times (0.32, -1.6) - 0.08 \times (-3.68, -12) = (0.4864, 0)$, so the two pulls on w_2 (inertia -0.96, gradient $+0.96$) cancel exactly. Step 3 is $(0.5473, 0.96)$ and $w = (-2.646, 0.36)$.

Figure `lec05-kp08-mom-steps`: table of the first three momentum steps.

t	gradient g	Δw	w after
1	(−4.00, 20.0)	(0.3200, −1.60)	(−3.680, −0.60)
2	(−3.68, −12.0)	(0.4864, 0.00)	(−3.194, −0.60)
3	(−3.19, −12.0)	(0.5473, 0.96)	(−2.646, 0.36)

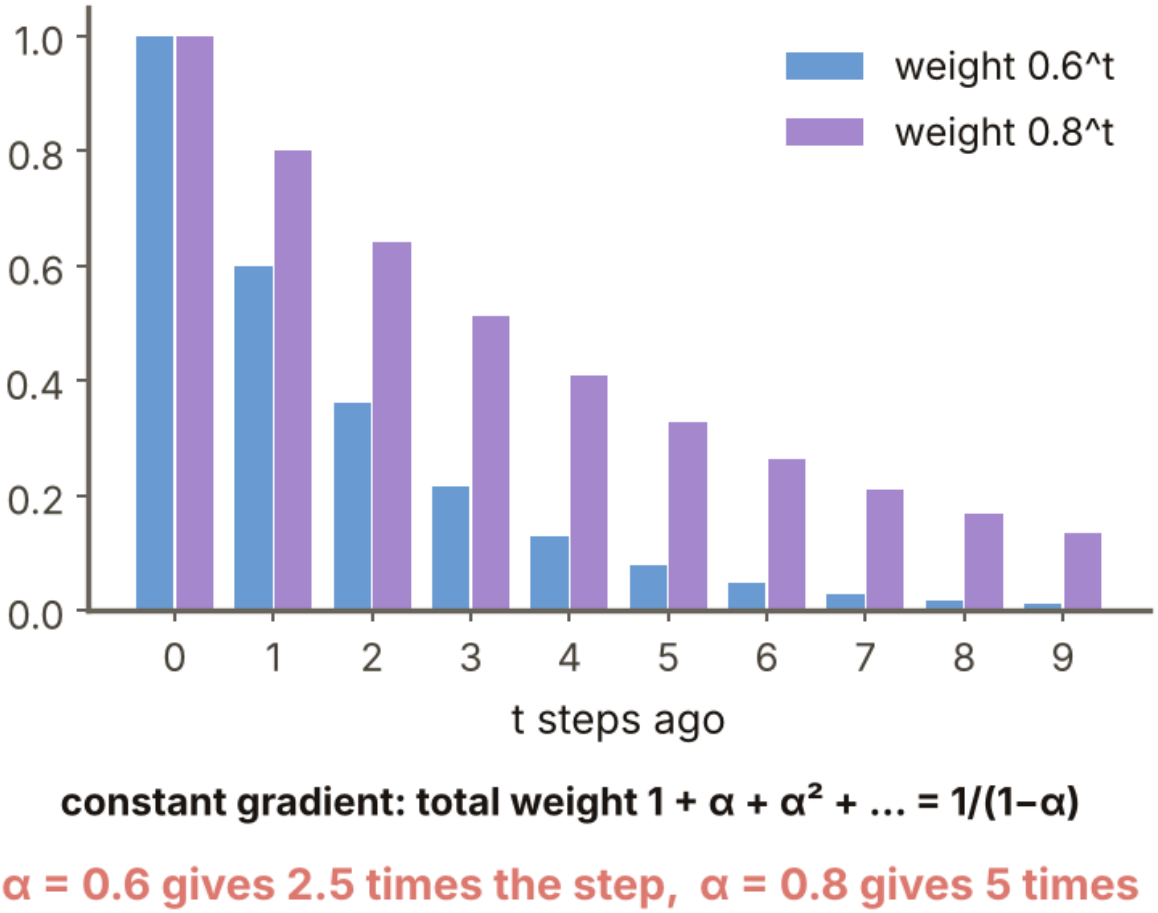
Δw(t) = 0.6·Δw(t−1) − 0.08·g(t), w starts at (−4, 1)

4. Why it works: an exponentially weighted average of past gradients

Exponentially weighted average: the newest gradient weighs most and every older one is multiplied by α once more, weights $1, \alpha, \alpha^2, \ldots$.

Unrolled over T steps: $\Delta w(T) = -\mu \sum_t \alpha^t g(T-t) + \alpha^T \Delta w(0)$ and the last term is near 0 after a few steps. With an unchanging gradient (the flat direction) the total weight is $1 + \alpha + \alpha^2 + \cdots = 1/(1-\alpha)$, which enlarges the learning rate by $1/(1-\alpha)$: 2.5 times for $\alpha = 0.6$ and 5 times for $\alpha = 0.8$ (checked with a fixed gradient over 80 steps). When the gradient alternates in sign (the steep direction) the weighted average cancels it. One mechanism, fast where it can be and steady where it must be.

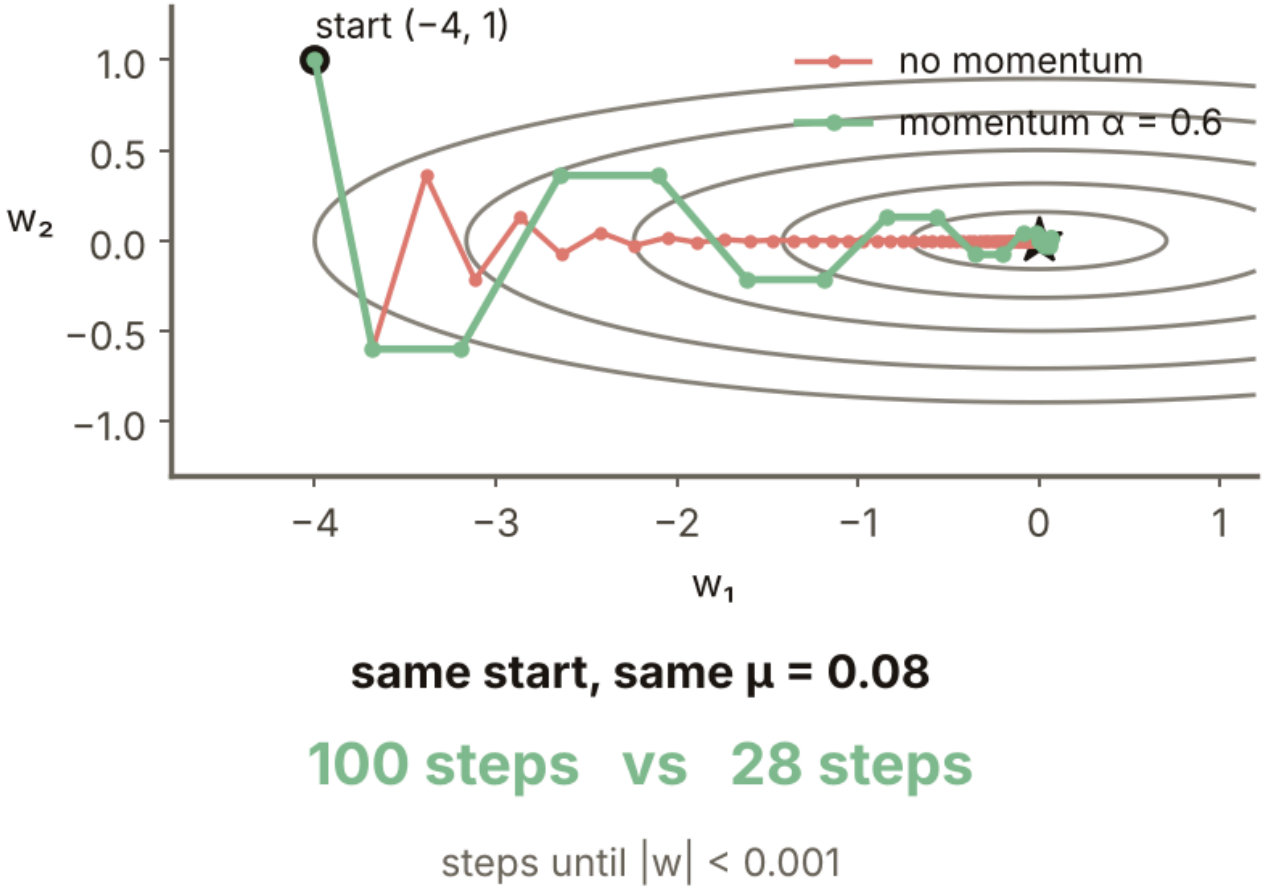
Figure `lec05-kp08-expo`: weights 0.6^t and 0.8^t and the total weight under a fixed gradient.



5. Two paths

Same start and same $\mu = 0.08$: plain needs 100 steps and momentum with $\alpha = 0.6$ needs 28 (program). The red path swings along w_2 many times while the green one shrinks the swing faster and also moves faster along w_1. The price is one more stored array for the previous update and one more knob α.

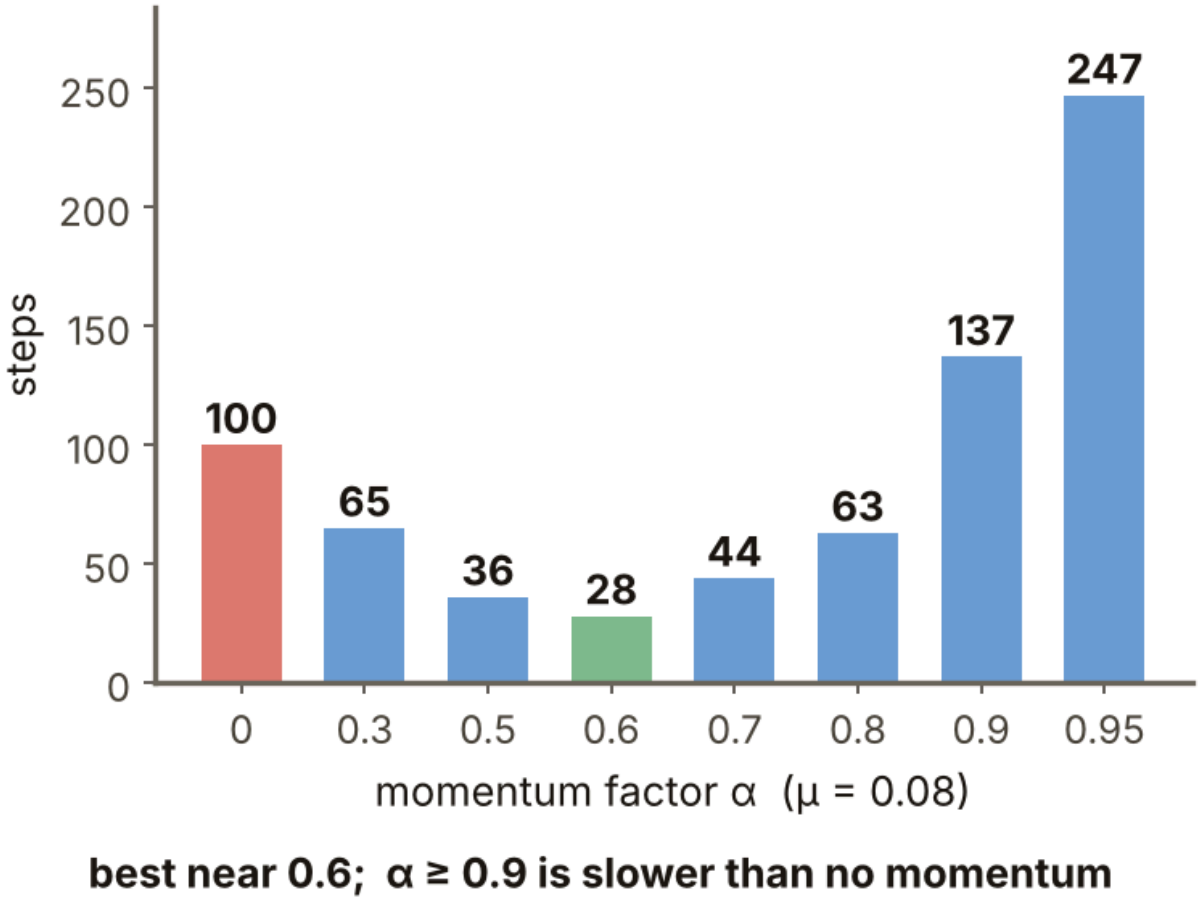
Figure `lec05-kp08-two-paths`: comparison of the two paths.



6. The best α

Common misconception: a larger α is always better. With $\mu = 0.08$, $\alpha = 0, 0.3, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95$ need 100, 65, 36, 28, 44, 63, 137 and 247 steps. $\alpha \geq 0.9$ is slower than no momentum: too much inertia overshoots the valley floor and rocks like a pendulum before it stops. In practice 0.1 to 0.8 is common, the best value depends on the problem and must be tried, and the source reports speed-ups of 2 times or more.

Figure `lec05-kp08-alpha-curve`: steps needed for each α.



7. Adaptive learning rate

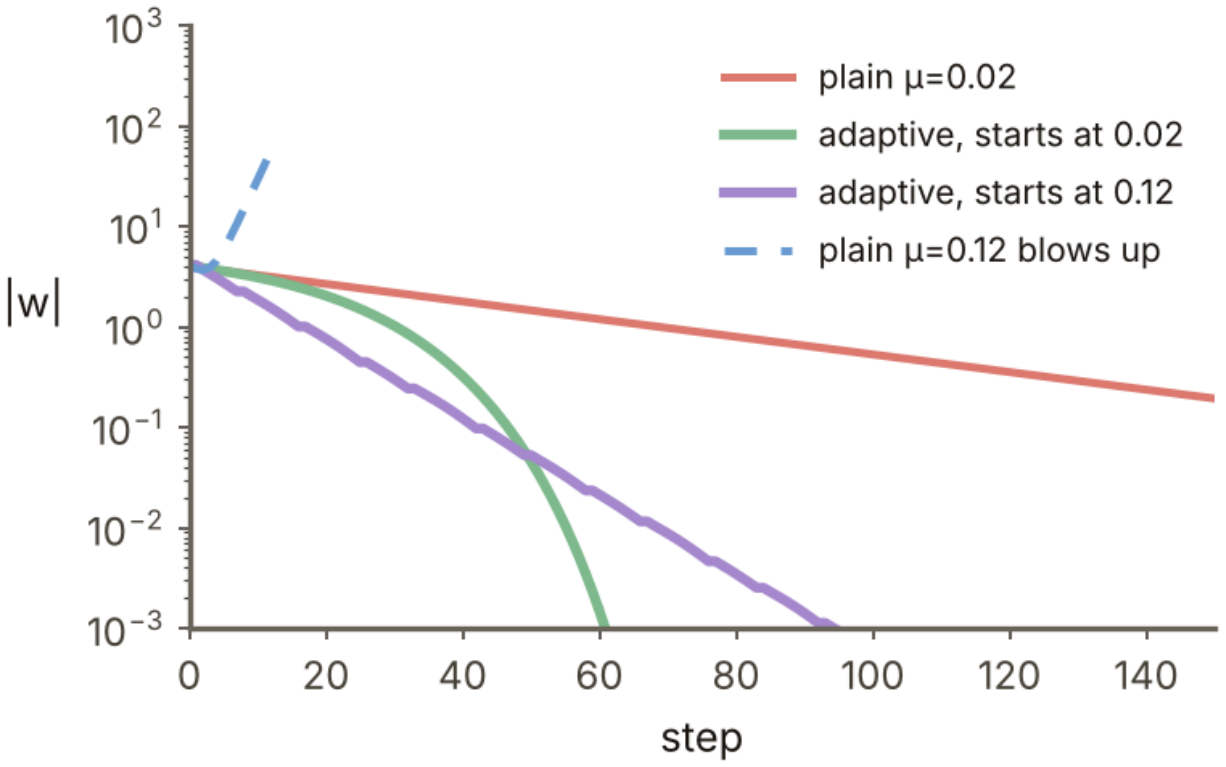
Adaptive learning rate: adjust μ automatically from the ratio of the cost before and after a step:

$$\mu(t) = \begin{cases} 1.05\,\mu(t-1), & J(t)/J(t-1) < 1 \\ 0.7\,\mu(t-1), & J(t)/J(t-1) > 1.04 \\ \mu(t-1), & \text{otherwise} \end{cases}$$

Typical values $r_i = 1.05$, $r_d = 0.7$, $c = 1.04$ (source). The rule is asymmetric: slow growth when better, hard braking when worse; when the cost rises this lecture rejects the step and zeroes the momentum (one of the source's suggestions).

Results (program): from $\mu_0 = 0.02$ plain needs 411 steps and adaptive 61; from $\mu_0 = 0.12$ plain blows up while adaptive rejects 12 steps and converges in 95; from $\mu_0 = 0.08$, already near the best, adaptive needs 93 steps, hardly better than plain 100.

Figure `lec05-kp08-adaptive`: semi-log curve of $|w|$ against steps.



steps: plain 0.02 → 411, adaptive 0.02 → 61, adaptive 0.12 → 95

the 0.12 run rejected 12 steps and cut μ by 30% each time

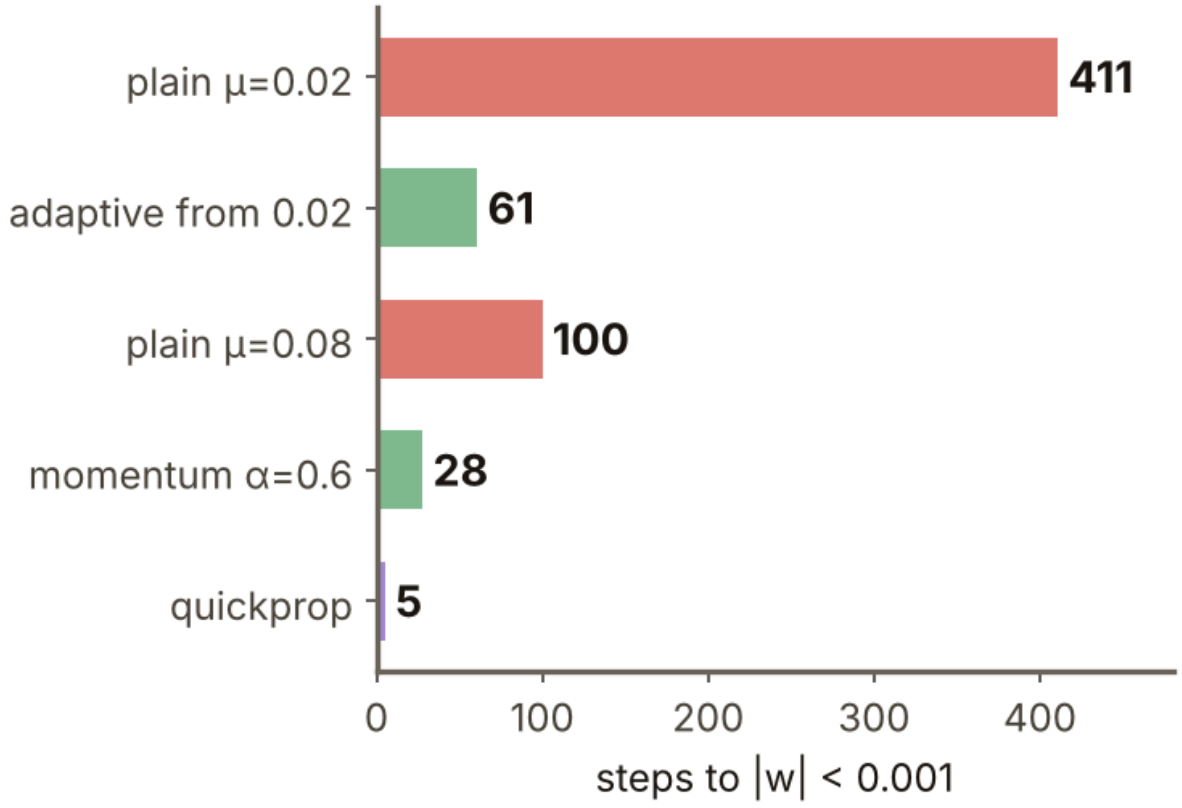
8. quickprop and a comparison of five methods

quickprop: treat every weight as independent; the last two slopes estimate the bottom of a parabola and the weight jumps there:

$$\Delta w(t) = \alpha(t)\,\Delta w(t-1), \qquad \alpha(t) = \min\left\{ \frac{g(t)}{g(t-1) - g(t)},\ \alpha_{\max} \right\},\ \alpha_{\max} \approx 1.75.$$

Example (w_2): step 1 is an ordinary step $1 \to -0.6$ with slope $20 \to -12$; $\alpha = -12/(20+12) = -0.375$ and $\Delta = -0.375 \times (-1.6) = +0.6$, so step 2 lands exactly on 0. The whole quadratic example takes 5 steps (plain $\mu = 0.02$ 411, adaptive 61, plain $\mu = 0.08$ 100, momentum 28). But every slice of this example is exactly a parabola, ideal for quickprop; real costs are not quadratic. Every speed-up costs extra complexity.

Figure `lec05-kp08-compare`: steps of the five methods.

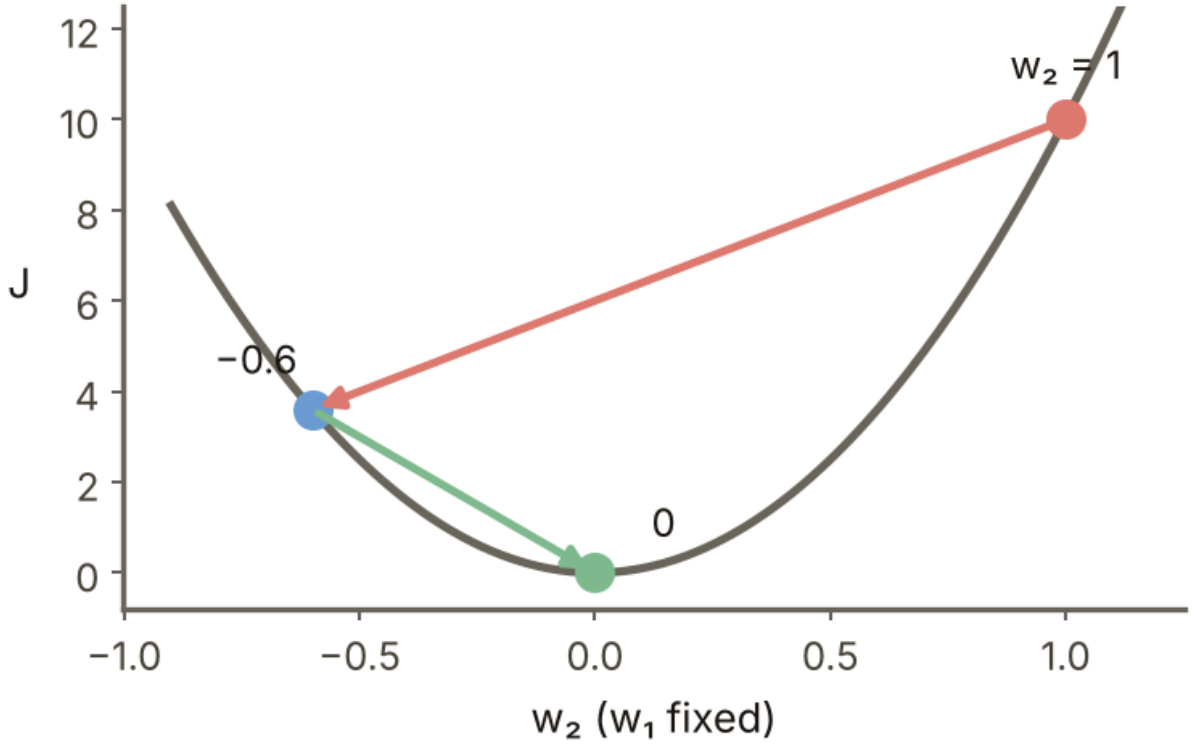


every speed-up costs extra memory or bookkeeping

Extension: Newton's method, practice and worked solution

Newton's method: use the curvature (the Hessian matrix H) to compute the valley floor directly, $\Delta w = -H^{-1}g$, and a quadratic is solved in one step. Here $H = \mathrm{diag}(1, 20)$ and $\Delta w = -w$, one step to the origin; the price is computing, storing and inverting the Hessian. quickprop can be seen as a one-dimensional Newton method per weight. The source also lists conjugate gradient, the Newton family, Kalman filtering and Levenberg–Marquardt, mostly needing Hessian elements; delta-delta and delta-bar-delta give every weight its own learning rate, grown when its gradient keeps its sign and shrunk when it flips.

Figure `lec05-kp08-secant`: the parabola along w_2 and the quickprop jump.



step 1: ordinary step 1 → −0.6 (μ = 0.08)

step 2: α = −12 / (20 − (−12)) = −0.375, Δ = −0.375 × (−1.6) = +0.6

lands exactly on the minimum of the parabola

Practice: the same valley, $\mu = 0.08$ and $\alpha = 0.6$. After step 2 $w = (-3.1936, -0.6)$ and the update of step 2 is $(0.4864, 0)$.

1. What is the gradient $g = (w_1, 20w_2)$ here?
2. What are the update $\Delta w(3) = 0.6\,\Delta w(2) - 0.08\,g(3)$ and the new w after step 3?

Figure `lec05-kp08-practice`: the numbers of the first two steps and the task.

$$J = \tfrac{1}{2}(w_1^2 + 20 \cdot w_2^2),\ \mu = 0.08,\ \alpha = 0.6$$

after step 1: $w = (-3.68, -0.6)$, $\Delta w = (0.32, -1.6)$

after step 2: $w = (-3.1936, -0.6)$, $\Delta w = (0.4864, 0.0)$

step 3: find g, Δw and w

$g = (w_1, 20 \cdot w_2)$ $\Delta w(3) = 0.6 \cdot \Delta w(2) - 0.08 \cdot g$

Worked solution of questions 1 and 2: (1) $g = (-3.1936,\ 20 \times (-0.6)) = (-3.1936, -12)$. (2) $\Delta w(3) = 0.6 \times (0.4864, 0) - 0.08 \times (-3.1936, -12) = (0.29184 + 0.25549,\ 0.96) = (0.54733, 0.96)$; $w = (-3.1936 + 0.5473,\ -0.6 + 0.96) = (-2.6463, 0.36)$. (3) Check: w_2 did not move in step 2, so its inertia is 0 in this step and the whole 0.96 of the gradient part becomes displacement.

Figure `lec05-kp08-practice-sol`: the terms of step 3.

$g = (w_1, 20 \cdot w_2)$	$= (-3.1936, -12.0)$
$0.6 \cdot \Delta w(2)$	$= (0.29184, 0.0)$
$-0.08 \cdot g$	$= (0.25549, 0.96)$
$\Delta w(3)$	$= (0.54733, 0.96)$
w after step 3	$= (-2.6463, 0.36)$

w_2 did not move in step 2, so its inertia term is 0

Terms and sources

English	中文
Curvature	曲率
Narrow valley (ill-conditioned surface)	狹長谷底（病態代價面）
Momentum	動量
Exponentially weighted average	指數加權平均
Adaptive learning rate	自適應學習率
quickprop	quickprop
Newton's method	牛頓法

Source: Chapter 4 KP08, §4.7 (read 2026-10-25). The momentum rule, the $1/(1-\alpha)$ derivation, the adaptive learning-rate rule and typical values, the quickprop formula and the Newton family follow the source; the example $J = \frac{1}{2}(w_1^2 + 20w_2^2)$, its start

and every step count and path were designed and recomputed for this lecture ([added], (program)).